

# SMART CAMPUS SDN ARCHITECTURE



by

|                |           |
|----------------|-----------|
| Mohsin Rasul   | BCY243024 |
| Muhammad Usman | BCY243033 |
| Ayisha Rehmat  | BCY243045 |
| Junaid Ahmed   | BCY243053 |
| Hamees Ahmed   | BCY243102 |

A Project Report submitted to the  
**DEPARTMENT OF ELECTRICAL ENGINEERING**  
in partial fulfillment of the requirements for the degree of  
**BACHELOR OF CYBER SECURITY**

Faculty of Engineering  
**Capital University of Science & Technology, Islamabad**

June, 2026

# SMART CAMPUS SDN ARCHITECTURE



by

|                |           |
|----------------|-----------|
| Mohsin Rasul   | BCY243024 |
| Muhammad Usman | BCY243033 |
| Ayisha Rehmat  | BCY243045 |
| Junaid Ahmed   | BCY243053 |
| Hamees Ahmed   | BCY243102 |

A Project Report submitted to the  
**DEPARTMENT OF ELECTRICAL ENGINEERING**  
in partial fulfillment of the requirements for the degree of  
**BACHELOR OF CYBER SECURITY**

Faculty of Engineering  
**Capital University of Science & Technology, Islamabad**

June, 2026

Copyright © 2026 by CUST Students

All rights reserved. Reproduction in whole or in part in any form requires the prior written permission of Mohsin Rasul, Junaid Ahmed, Hamees Ahmed, Muhammad Usman, Ayisha Rehmat or designated representatives.

# DECLARATION

It is declared that this is an original piece of our own work, except where otherwise acknowledged in text and references. This work has not been submitted in any form for another degree or diploma at any university or other institution for tertiary education and shall not be submitted by us in future for obtaining any degree from this or any other University or Institution.

**Mohsin Rasul**

Reg. No. BCY243024

**Junaid Ahmed**

Reg. No. BCY243053

**Hamees Ahmed**

Reg. No. BCY243102

**Muhammad Usman**

Reg. No. BCY243033

**Ayisha Rehmat**

Reg. No. BCY243045

June, 2026

## CERTIFICATE OF APPROVAL

It is certified that the project titled **Smart Campus SDN Architecture** carried out by the designated group members under the supervision of **Dr. Jehan Zeb** and **Ms. Ayesha Riaz**, Capital University of Science & Technology, Islamabad, is fully adequate, in scope and in quality, as a Complex Computing Problem for the course **CY3711: Computer Communication and Networks**.

Course Instructor: \_\_\_\_\_

**Dr. Jehan Zeb**

Lab Instructor: \_\_\_\_\_

**Ms. Ayesha Riaz**

# ABSTRACT

Modern academic campuses rely on increasingly complex networks to support educational and administrative operations. Traditional flat Layer-2 network architectures suffer from severe broadcast congestion, administrative limitations, and inadequate security segmentation. This project systematically identifies these vulnerabilities and proposes a scalable, SDN-enabled Three-Tier campus architecture.

Utilizing the Mininet network emulator and a Ryu OpenFlow controller, a structured Core-Distribution-Access hierarchy was developed. The redesign integrates strict 802.1Q VLAN segmentation to isolate departmental traffic alongside centralized DHCP services. Security enforcement was achieved by pushing dynamic Access Control List (ACL) policies directly to the switch hardware via the SDN controller, completely neutralizing unauthorized inter-VLAN movement. Through comprehensive Wireshark packet analysis, the functional integrity and mechanics of ARP, DNS, DHCP, OSPF, and TCP handshake protocols were interpreted in detail. Furthermore, OpenFlow traffic statistics were utilized as a centralized monitoring mechanism. The resulting infrastructure provides real-time visibility, robust threat mitigation, and intelligent traffic engineering, directly advancing the United Nations Sustainable Development Goals SDG-9 and SDG-11.

# TABLE OF CONTENTS

|                                                                               |            |
|-------------------------------------------------------------------------------|------------|
| <b>Declaration</b>                                                            | <b>iii</b> |
| <b>Certificate of Approval</b>                                                | <b>iv</b>  |
| <b>Abstract</b>                                                               | <b>v</b>   |
| <b>List of Acronyms</b>                                                       | <b>x</b>   |
| <b>1 BASELINE ANALYSIS AND GAPS</b>                                           | <b>1</b>   |
| 1.1 Assumed Flat Baseline Network and Limitations . . . . .                   | 1          |
| 1.2 Gap Analysis: Efficiency, Security, Scalability, and Administration . . . | 1          |
| <b>2 ARCHITECTURE AND ROUTING DESIGN</b>                                      | <b>3</b>   |
| 2.1 Proposed Hierarchical Core-Distribution-Access Architecture . . . . .     | 3          |
| 2.2 VLAN Segmentation and Subnet Addressing Plans . . . . .                   | 4          |
| 2.3 Inter-VLAN Routing and Gateway Addressing Logic . . . . .                 | 5          |
| 2.4 Routing Protocol Recommendation . . . . .                                 | 5          |
| <b>3 SOFTWARE DEFINED NETWORKING (SDN)</b>                                    | <b>6</b>   |
| 3.1 SDN Concepts for Centralized Management . . . . .                         | 6          |
| 3.2 SDN Improvements in Administration, Security, and Traffic Engineering     | 6          |
| 3.3 Centralized Network Monitoring and Traffic Visualization . . . . .        | 7          |
| <b>4 SECURITY, ACLS, AND SUSTAINABILITY</b>                                   | <b>8</b>   |
| 4.1 Comprehensive CIA Triad Security Justification . . . . .                  | 8          |
| 4.2 Identified Network Threat: ARP Spoofing . . . . .                         | 9          |
| 4.3 ACL Rule Restricting Unauthorized Inter-VLAN Access . . . . .             | 9          |
| 4.4 Sustainability and Smart-Campus Benefits (SDG-9 and SDG-11) . . . .       | 10         |
| <b>5 SIMULATION RESULTS AND PROTOCOL ANALYSIS</b>                             | <b>11</b>  |
| 5.1 Initialization and Topology Setup . . . . .                               | 11         |
| 5.2 Host IP Allocation . . . . .                                              | 13         |
| 5.3 SDN Policy Enforcement Verification . . . . .                             | 14         |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| 5.4      | Network Traffic Capture and Protocol Interpretation ..... | 14        |
| 5.4.1    | DHCP (Dynamic Host Configuration Protocol).....           | 15        |
| 5.4.2    | TCP Retransmission and Firewall Verification .....        | 16        |
| 5.4.3    | ARP (Address Resolution Protocol).....                    | 17        |
| 5.4.4    | TCP Three-Way Handshake.....                              | 18        |
| 5.4.5    | DNS (Domain Name System).....                             | 20        |
| 5.4.6    | OSPF Routing Verification .....                           | 20        |
| <b>6</b> | <b>CONCLUSION AND FUTURE WORK</b>                         | <b>23</b> |
| <b>A</b> | <b>SOURCE CODE</b>                                        | <b>25</b> |
| A.1      | Ryu SDN Controller Script (customRyu.py) .....            | 25        |
| A.2      | Mininet Topology Script (campusNetwork.py) .....          | 27        |



# List of Figures

|      |                                                                                   |    |
|------|-----------------------------------------------------------------------------------|----|
| 1.1  | Baseline Architecture: Flat L2 Network Diagram .....                              | 1  |
| 2.1  | Proposed SDN-Integrated Three-Tier Campus Architecture .....                      | 4  |
| 5.1  | Initialization of Ryu Controller and ACL Rule Application .....                   | 11 |
| 5.2  | Execution of Mininet Topology Script.....                                         | 12 |
| 5.3  | 802.1Q VLAN Tagging enforced on OVS switches .....                                | 12 |
| 5.4  | Centralized DHCP Service Initialization .....                                     | 12 |
| 5.5  | Centralized Monitoring: Open vSwitch Flow Tables showing Priority Drop Rules..... | 13 |
| 5.6  | Host 1 (ECE) IP Allocation in VLAN 10 .....                                       | 13 |
| 5.7  | Host 2 (CS) IP Allocation in VLAN 20.....                                         | 13 |
| 5.8  | Host 3 (Admin) IP Allocation in VLAN 30.....                                      | 13 |
| 5.9  | Host 4 (Labs) IP Allocation in VLAN 40.....                                       | 14 |
| 5.10 | ICMP Connectivity Blocked by SDN Firewall.....                                    | 14 |
| 5.11 | Flow Table Verification of Packet Interception .....                              | 14 |
| 5.12 | Terminal Execution of DHCP Client Renewal .....                                   | 15 |
| 5.13 | Wireshark Capture: DHCP DORA Sequence.....                                        | 16 |
| 5.14 | Terminal output showing TCP Connection Timeout .....                              | 17 |
| 5.15 | Wireshark Capture: TCP Retransmission due to SDN Drop Rule.....                   | 17 |
| 5.16 | Flushing ARP Cache and initiating traffic.....                                    | 18 |
| 5.17 | Wireshark Capture: ARP Request and Reply .....                                    | 18 |
| 5.18 | Successful HTTP Request between authorized subnets.....                           | 19 |
| 5.19 | Wireshark Capture: TCP Three-Way Handshake .....                                  | 19 |
| 5.20 | Generating a DNS Request to the Local Resolver .....                              | 20 |
| 5.21 | Wireshark Capture: DNS Query and Response .....                                   | 20 |
| 5.22 | Verification of Active OSPF Process via vtysh.....                                | 21 |
| 5.23 | OSPF Link State and Area 0 Verification.....                                      | 22 |
| 5.24 | FRR Running Configuration confirming OSPF Integration.....                        | 22 |

# List of Tables

|     |                                              |   |
|-----|----------------------------------------------|---|
| 2.1 | VLAN Segmentation and Addressing Logic ..... | 4 |
|-----|----------------------------------------------|---|

## LIST OF ACRONYMS

|      |                                     |
|------|-------------------------------------|
| ACL  | Access Control List                 |
| ARP  | Address Resolution Protocol         |
| DHCP | Dynamic Host Configuration Protocol |
| DNS  | Domain Name System                  |
| MAC  | Media Access Control                |
| OSPF | Open Shortest Path First            |
| SDN  | Software-Defined Networking         |
| SDG  | Sustainable Development Goal        |
| TCP  | Transmission Control Protocol       |
| VLAN | Virtual Local Area Network          |

# CHAPTER 1

## BASELINE ANALYSIS AND GAPS

### 1.1 Assumed Flat Baseline Network and Limitations

Before implementing a modern architecture, it is essential to analyze the existing infrastructure. The assumed baseline campus infrastructure operates as a "flat" Layer-2 network. In this topology, there are no logical subdivisions; all access switches and distribution devices connect directly into a single, massive broadcast domain utilizing a single IP subnet.

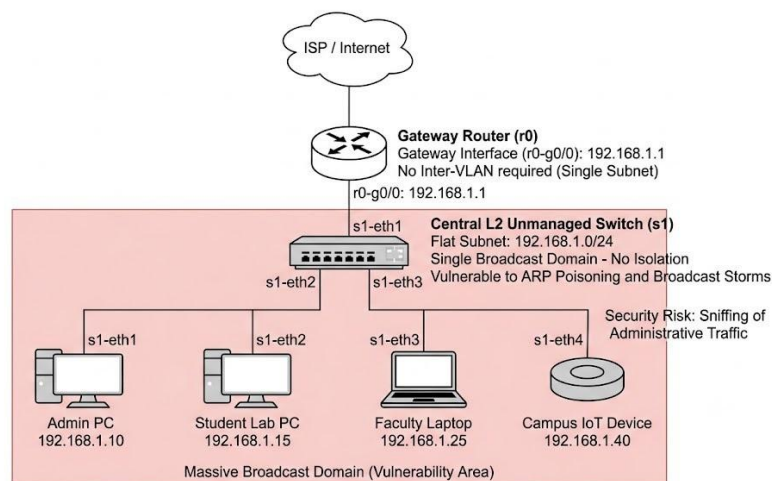


Figure 1.1: Baseline Architecture: Flat L2 Network Diagram

The primary limitation of this baseline is that every device on the campus is treated equally on the data link layer. When a host sends a broadcast frame (such as an ARP request to find a MAC address), that frame is mathematically flooded out of every active port on every switch across the entire campus.

### 1.2 Gap Analysis: Efficiency, Security, Scalability, and Administration

The comprehensive analysis of the flat baseline network reveals critical operational and administrative gaps that necessitate a redesign:

- **Efficiency Gaps (Broadcast Storms):** In a flat network, broadcast traffic grows exponentially as new devices are added. Background traffic consumes vital network bandwidth and interrupts the CPU of every connected host, meaning high-throughput data in the student labs degrades the performance of the administration servers.
- **Security Gaps:** Because all hosts reside on the same Layer-2 subnet, there are no inherent security boundaries. A compromised machine in a public student laboratory has direct, unfiltered access to confidential faculty and administrative machines, enabling trivial packet sniffing and lateral movement.
- **Scalability Gaps:** Expanding the network requires expanding the already congested broadcast domain. Eventually, the sheer volume of background traffic will cause the network to collapse, severely limiting the physical growth of the campus.
- **Administrative Gaps:** Traditional networks lack centralized control. If an administrator needs to block a specific type of traffic, they must manually log into dozens of individual switches via SSH to configure local access lists. This is prone to human error and drastically slows down incident response times.

## CHAPTER 2

### ARCHITECTURE AND ROUTING DESIGN

#### 2.1 Proposed Hierarchical Core-Distribution-Access Architecture

To systematically resolve the bottlenecks identified in the baseline network, the re-designed campus adheres to the Cisco Three-Tier hierarchical model. This structured approach divides the network into functional layers, localizing traffic and simplifying management.

- **Core Layer:** A high-speed backbone acting as the primary gateway. Its sole purpose is to route packets as quickly as possible without getting bogged down by complex packet filtering.
- **Distribution Layer:** This layer aggregates connections from the access switches. It serves as the boundary between Layer 2 (switching) and Layer 3 (routing), acting as the primary point for enforcing network policies and containing broadcast domains.
- **Access Layer:** The lowest tier provides direct connectivity to end-user devices (PCs, laptops, IoT sensors) and applies initial port security and VLAN tagging.

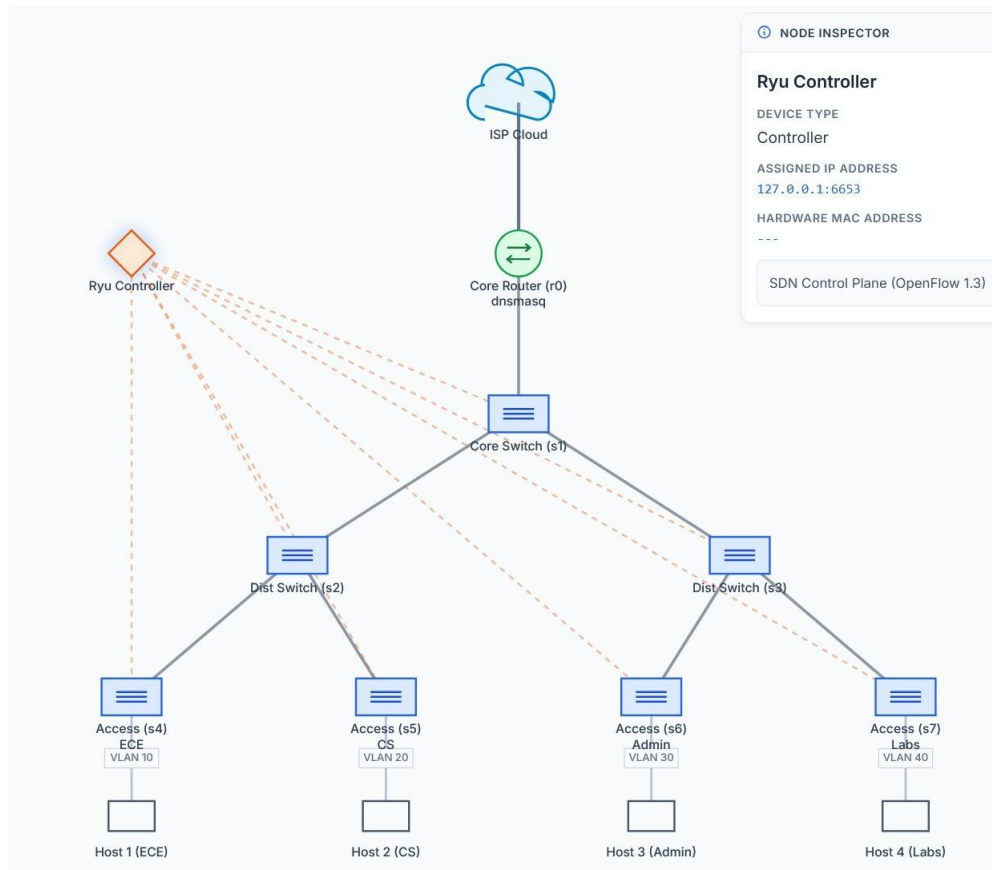


Figure 2.1: Proposed SDN-Integrated Three-Tier Campus Architecture

## 2.2 VLAN Segmentation and Subnet Addressing Plans

To break up the massive broadcast domain, Virtual Local Area Networks (VLANs) utilizing the IEEE 802.1Q standard were implemented. VLANs logically group devices together as if they were on their own separate physical switches, regardless of their actual physical location on campus.

Table 2.1: VLAN Segmentation and Addressing Logic

| VLAN ID | Department Name | Network Address | Subnet Mask   | Gateway      |
|---------|-----------------|-----------------|---------------|--------------|
| 10      | ECE Department  | 192.168.10.0/24 | 255.255.255.0 | 192.168.10.1 |
| 20      | CS Department   | 192.168.20.0/24 | 255.255.255.0 | 192.168.20.1 |
| 30      | Administration  | 192.168.30.0/24 | 255.255.255.0 | 192.168.30.1 |
| 40      | Student Labs    | 192.168.40.0/24 | 255.255.255.0 | 192.168.40.1 |

## 2.3 Inter-VLAN Routing and Gateway Addressing Logic

By definition, devices in different VLANs cannot communicate directly with each other at Layer 2. To facilitate authorized communication between isolated departments, Inter-VLAN routing was implemented at the Core Router using "Router-on-a-Stick" logic.

When a host in VLAN 10 wishes to communicate with a server in VLAN 30, it sends the packet to its default gateway (192.168.10.1). The core router receives the frame on a trunked interface, strips the 802.1Q VLAN 10 tag, inspects the Layer 3 destination IP, logically routes it to the VLAN 30 sub-interface (192.168.30.1), re-tags the frame with VLAN 30, and forwards it down to the destination.

## 2.4 Routing Protocol Recommendation

For a hierarchical campus design of this scale, **Open Shortest Path First (OSPF)** is the recommended dynamic routing protocol.

**Justification:** OSPF is a sophisticated link-state protocol. Unlike older distance-vector protocols (such as RIP) that rely on simple hop counts, OSPF uses Dijkstra's Shortest Path First algorithm to calculate the absolute fastest route based on link bandwidth and cost. Furthermore, OSPF offers superior scalability by dividing the campus into logical "Areas" (with the core operating as Area 0). This prevents routing updates in a remote campus building from unnecessarily recalculating the entire core routing table, resulting in nearly instantaneous convergence if a distribution link fails.



## CHAPTER 3

# SOFTWARE DEFINED NETWORKING (SDN)

### 3.1 SDN Concepts for Centralized Management

Traditional network hardware relies on distributed intelligence, meaning every individual switch must independently learn MAC addresses and calculate forwarding paths using its own internal processor. Software-Defined Networking (SDN) revolutionizes this paradigm by decoupling the control plane (the "brain" that makes routing decisions) from the data plane (the physical hardware that forwards packets).

In this redesigned campus, a centralized Ryu controller acts as the supreme intelligence for the entire network. Utilizing the OpenFlow protocol, the controller communicates directly with the OVS (Open vSwitch) instances, dynamically injecting flow tables into the switches to dictate exactly how packets should be handled.

### 3.2 SDN Improvements in Administration, Security, and Traffic Engineering

Integrating SDN into the campus architecture drastically improves operational efficiency:

- **VLAN Administration and Automated Configuration:** Network administrators can dynamically provision, modify, or tear down VLANs across the entire campus via a single Python script on the controller. This automated configuration eliminates the need to manually SSH into multiple access switches, drastically reducing human error.
- **Security and Policy Enforcement:** The controller acts as a distributed firewall. Instead of buying expensive hardware firewalls for every building, the SDN controller pushes deterministic access rules directly to the core switches, dropping malicious traffic at the hardware level before it can enter the campus backbone.
- **Intelligent Traffic Engineering:** Because the controller maintains a global view of the entire network topology, it can intelligently reroute traffic during peak utilization periods to prevent bandwidth congestion.

### **3.3 Centralized Network Monitoring and Traffic Visualization**

Effective network management requires deep visibility into traffic behavior. While enterprise environments may use dedicated GUI dashboards, the core underlying mechanism for SDN traffic visualization relies on continuous polling of OpenFlow flow statistics.

Within the simulated environment, centralized monitoring was implemented by executing `ovs-ofctl dump-flows` commands directly against the core switches. This mechanism provides real-time traffic statistics, capturing the exact number of packets (`n_packets`) and bytes (`n_bytes`) that match specific flow rules. By correlating these flow statistics with Wireshark's packet capture graphs, administrators can successfully visualize bandwidth utilization, monitor active host communication patterns, and detect anomalous traffic spikes without deploying complex third-party agents.

## CHAPTER 4

# SECURITY, ACLS, AND SUSTAINABILITY

### 4.1 Comprehensive CIA Triad Security Justification

A modern smart campus must be engineered with security as a foundational element, not an afterthought. The proposed SDN-enabled architecture fulfills the core pillars of the CIA Triad through structural isolation and programmatic control.

- **Confidentiality (Protecting Data from Unauthorized Disclosure):** In an academic environment, preserving the privacy of student records, faculty research, and administrative communications is paramount. The redesign enforces confidentiality through strict 802.1Q VLAN segmentation. By physically constraining broadcast domains, students operating in the Lab network (VLAN 40) are mathematically prevented from utilizing promiscuous packet-sniffing tools (like Wireshark) to capture or eavesdrop on unencrypted traffic originating from the Administration network (VLAN 30).
- **Integrity (Preventing Unauthorized Modification):** Network integrity ensures that data reaches its intended destination without being altered, and that the network pathways themselves remain untampered. In a legacy network, an attacker can manipulate spanning-tree protocols or flood routing tables to alter traffic flows. By centralizing network intelligence into the Ryu SDN controller, the architecture establishes a Single Point of Truth. The data plane switches are stripped of their autonomous decision-making power; they only forward packets based on the strictly authenticated OpenFlow rules pushed by the controller, effectively immunizing the network against localized switch-level tampering.
- **Availability (Ensuring Reliable Access to Resources):** A secure network is useless if legitimate users cannot access it. Flat networks are notoriously susceptible to self-inflicted denial-of-service via broadcast storms. The hierarchical Core-Distribution-Access model ensures high availability by containing broadcast traffic to individual access layers, preventing a faulty device in one department from degrading the entire campus core. Furthermore, dynamic routing guarantees that if a primary distribution link is severed, traffic is seamlessly rerouted to maintain

## 4.2 Identified Network Threat: ARP Spoofing

**Threat Mechanics:** Address Resolution Protocol (ARP) spoofing is a severe Layer 2 cyberattack. In a standard flat network, devices trust all ARP replies inherently. A malicious actor can broadcast forged ARP packets, falsely claiming that their device's physical MAC address belongs to the IP address of the campus default gateway. Consequently, all computers on the network will begin sending their external internet traffic directly to the attacker's machine, enabling a devastating Man-in-the-Middle (MitM) attack.

**Mitigation:** The redesigned architecture drastically neutralizes this threat. Because VLANs segment the campus into isolated broadcast domains, an attacker in the Student Labs attempting to ARP spoof can only disrupt the local Lab VLAN. They are physically unable to poison the ARP caches of the Administration or ECE departments, effectively quarantining the threat to a single, low-priority subnet.

## 4.3 ACL Rule Restricting Unauthorized Inter-VLAN Access

To enforce the principle of least privilege and establish a zero-trust boundary, Access Control Lists (ACLs) were programmed directly into the core switch via the SDN controller.

**Targeted Scenario:** Administrative security policies dictate that devices in the public Student Labs must not initiate communication with the highly sensitive Administration subnet.

- **Source Network:** Student Labs (VLAN 40) → 192.168.40.0/24
- **Destination Network:** Administration (VLAN 30) → 192.168.30.0/24
- **Action:** DROP
- **OpenFlow Priority Level:** 40000 (Overrides standard routing)
- **Implementation Logic:**  
priority=40000, ip, nw\_src=192.168.40.0/24, nw\_dst=192.168.30.0/24,  
actions=drop

## **4.4 Sustainability and Smart-Campus Benefits (SDG-9 and SDG-11)**

The redesign of the campus network strongly aligns with the United Nations Sustainable Development Goals by transforming traditional hardware into an adaptable, eco-friendly smart campus.

### **SDG-9: Industry, Innovation, and Infrastructure**

SDN modernizes physical infrastructure by decoupling software from proprietary hardware. The university builds a resilient digital backbone where network behaviors are programmed dynamically via software APIs rather than by purchasing new physical routing equipment. This reduces electronic waste (e-waste) and capital expenditures, allowing the institution to innovate and scale its infrastructure sustainably as new technologies emerge.

### **SDG-11: Sustainable Cities and Communities**

A modern university campus operates as a microcosm of a smart city. The implementation of strict VLAN segmentation optimizes power consumption across the community. By isolating IoT devices (such as smart lighting, HVAC temperature controllers, and security cameras) into their own domains, the network prevents excessive background broadcast noise from keeping edge-device radios continuously active. This intelligent traffic management significantly reduces the overall energy footprint of the campus ecosystem while ensuring critical community safety services remain uninterrupted.

## CHAPTER 5

# SIMULATION RESULTS AND PROTOCOL ANALYSIS

### 5.1 Initialization and Topology Setup

The implementation began by instantiating the Ryu SDN controller within a Docker container. Figure 5.1 depicts the successful controller initialization, confirming connectivity to the OpenFlow switches and the immediate enforcement of the security ACL policies.

```
kuchbhi@MR-U-VH:~$ sudo docker run --network host -v /home/kuchbhi/Desktop:/apps osrg/ryu ryu-manager /apps/customRyu.py
loading app /apps/customRyu.py
loading app ryu.controller.ofp_handler
instantiating app ryu.controller.ofp_handler of OFPHandler
instantiating app /apps/customRyu.py of CustomCampusController
Switch fully connected and NORMAL flow pushed: 7
Switch fully connected and NORMAL flow pushed: 1
Applying CUST campus ACL policy to Core Switch (s1)...
ACL Rule 1 installed: Student_Labs(40) -> Admin(30) DROP
ACL Rule 2 installed: ECE_Dept(10) -> CS_Dept(20) DROP
Switch fully connected and NORMAL flow pushed: 6
Switch fully connected and NORMAL flow pushed: 4
Switch fully connected and NORMAL flow pushed: 3
Switch fully connected and NORMAL flow pushed: 5
Switch fully connected and NORMAL flow pushed: 2
Switch fully connected and NORMAL flow pushed: 1
Applying CUST campus ACL policy to Core Switch (s1)...
ACL Rule 1 installed: Student_Labs(40) -> Admin(30) DROP
ACL Rule 2 installed: ECE_Dept(10) -> CS_Dept(20) DROP
Switch fully connected and NORMAL flow pushed: 2
Switch fully connected and NORMAL flow pushed: 3
Switch fully connected and NORMAL flow pushed: 4
Switch fully connected and NORMAL flow pushed: 5
Switch fully connected and NORMAL flow pushed: 6
Switch fully connected and NORMAL flow pushed: 7
```

Figure 5.1: Initialization of Ryu Controller and ACL Rule Application

The Python Mininet script was executed to build the physical topology. Figures 5.2 and 5.3 demonstrate the successful instantiation of the virtual nodes and the application of 802.1Q tags across the designated access and trunk ports.

```
kuchbhi@MR-U-VM:~/Desktop$ sudo python3 campusNetwork.py
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3 h4 r0
*** Adding switches:
s1 s2 s3 s4 s5 s6 s7
*** Adding links:
(h1, s4) (h2, s5) (h3, s6) (h4, s7) (r0, s1) (r0, s1) (r0, s1) (r0, s1) (s1, s2) (s1, s3) (s2, s4) (s2, s5) (s3, s6) (s3, s7)
*** Configuring hosts
h1 h2 h3 h4 r0
*** Starting controller
c0
*** Starting 7 switches
s1 s2 s3 s4 s5 s6 s7 ...
```

Figure 5.2: Execution of Mininet Topology Script

```
*** [1/5] Core router interfaces are up

*** Applying 802.1Q VLAN tagging and enforcing OpenFlow 1.3...
- s1 : s1-eth1 -> ACCESS vlan10 (r0-eth1 / ECE)
- s1 : s1-eth2 -> ACCESS vlan20 (r0-eth2 / CS)
- s1 : s1-eth3 -> ACCESS vlan30 (r0-eth3 / Admin)
- s1 : s1-eth4 -> ACCESS vlan40 (r0-eth4 / Labs)
- s1 : s1-eth5 -> TRUNK (10,20,30,40) (s1->s2)
- s1 : s1-eth6 -> TRUNK (10,20,30,40) (s1->s3)
- s2 : s2-eth1 -> TRUNK (10,20,30,40) (s2->s1)
- s2 : s2-eth2 -> TRUNK (10,20,30,40) (s2->s4)
- s2 : s2-eth3 -> TRUNK (10,20,30,40) (s2->s5)
- s3 : s3-eth1 -> TRUNK (10,20,30,40) (s3->s1)
- s3 : s3-eth2 -> TRUNK (10,20,30,40) (s3->s6)
- s3 : s3-eth3 -> TRUNK (10,20,30,40) (s3->s7)
- s4 : s4-eth1 -> TRUNK (10,20,30,40) (s4->s2)
- s4 : s4-eth2 -> ACCESS vlan10 (s4->h1)
- s5 : s5-eth1 -> TRUNK (10,20,30,40) (s5->s2)
- s5 : s5-eth2 -> ACCESS vlan20 (s5->h2)
- s6 : s6-eth1 -> TRUNK (10,20,30,40) (s6->s3)
- s6 : s6-eth2 -> ACCESS vlan30 (s6->h3)
- s7 : s7-eth1 -> TRUNK (10,20,30,40) (s7->s3)
- s7 : s7-eth2 -> ACCESS vlan40 (s7->h4)

*** Waiting 5 seconds for Ryu Controller to synchronize with all switches...
```

Figure 5.3: 802.1Q VLAN Tagging enforced on OVS switches

Following the topology build, the centralized DHCP service (dnsmasq) was launched on the core router, isolating daemon instances to their respective VLAN interfaces (Figure 5.4).

```
*** [2/5] Starting centralized DHCP service on r0 (per-VLAN)...
- dnsmasq for ece VLAN bound to r0-eth1 (pid 15554)
- dnsmasq for cs VLAN bound to r0-eth2 (pid 15558)
- dnsmasq for labs VLAN bound to r0-eth4 (pid 15562)
*** [3/5] Assigning static service IP to Admin host (h3)...
*** [4/5] Requesting DHCP leases for h1 (ECE), h2 (CS), h4 (Labs)...
- h1 leased: inet 192.168.10.147/24 brd 192.168.10.255 scope global dynamic h1-eth0 (correct VLAN pool)
- h2 leased: inet 192.168.20.142/24 brd 192.168.20.255 scope global dynamic h2-eth0 (correct VLAN pool)
- h4 leased: inet 192.168.40.121/24 brd 192.168.40.255 scope global dynamic h4-eth0 (correct VLAN pool)

*** Network is up.
Verify ACL : sh ovs-ofctl -O OpenFlow13 dump-flows s1
```

Figure 5.4: Centralized DHCP Service Initialization

Figure 5.5 demonstrates the core switch flow table state. This acts as our centralized monitoring visualization, verifying that the Priority 40000 drop rules were successfully injected into the hardware pipeline alongside standard routing tables.

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
cookie=0x0, duration=45.032s, table=0, n_packets=0, n_bytes=0, priority=40000,ip,nw_src=192.168.40.0/24,nw_dst=192.168.30.0/24 actions=drop
cookie=0x0, duration=45.032s, table=0, n_packets=0, n_bytes=0, priority=40000,ip,nw_src=192.168.10.0/24,nw_dst=192.168.20.0/24 actions=drop
cookie=0x0, duration=45.032s, table=0, n_packets=111, n_bytes=18851, priority=0 actions=NORMAL
```

Figure 5.5: Centralized Monitoring: Open vSwitch Flow Tables showing Priority Drop Rules

## 5.2 Host IP Allocation

Figures 5.6 through 5.9 confirm that the automated IP provisioning succeeded perfectly. Each departmental host correctly requested and received an IP address uniquely corresponding to its assigned VLAN address block, validating the segmentation logic.

```
mininet> h1 ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: h1-eth0@if284: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether be:4c:f2:56:1b:fc brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.10.147/24 brd 192.168.10.255 scope global dynamic h1-eth0
        valid_lft 42753sec preferred_lft 42753sec
```

Figure 5.6: Host 1 (ECE) IP Allocation in VLAN 10

```
mininet> h2 ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: h2-eth0@if285: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether e6:ad:39:01:2e:90 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.20.142/24 brd 192.168.20.255 scope global dynamic h2-eth0
        valid_lft 42730sec preferred_lft 42730sec
```

Figure 5.7: Host 2 (CS) IP Allocation in VLAN 20

```
mininet> h3 ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: h3-eth0@if286: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 3a:52:28:15:61:fe brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.30.10/24 brd 192.168.30.255 scope global h3-eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::3852:28ff:fe15:61fe/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
```

Figure 5.8: Host 3 (Admin) IP Allocation in VLAN 30



```

mininet> h4 ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: h4-eth0@if287: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether aa:04:a2:3d:bb:2b brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.40.121/24 brd 192.168.40.255 scope global dynamic h4-eth0
        valid_lft 42600sec preferred_lft 42600sec

```

Figure 5.9: Host 4 (Labs) IP Allocation in VLAN 40

### 5.3 SDN Policy Enforcement Verification

To empirically verify the Zero-Trust configuration, an ICMP Ping request was generated from Host 4 in the Student Labs toward the Administration server. Figure 5.10 shows 100% packet loss, proving the SDN firewall is active and correctly interpreting boundaries.

```

mininet> h4 ping -c 3 192.168.30.10
PING 192.168.30.10 (192.168.30.10) 56(84) bytes of data.

--- 192.168.30.10 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2073ms

```

Figure 5.10: ICMP Connectivity Blocked by SDN Firewall

Figure 5.11 provides the mathematical evidence via flow monitoring: the `n_packets` counter on the OpenFlow drop rule incremented exactly to match the number of dropped ICMP echo requests (3 packets), confirming precise controller enforcement.

```

mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1
cookie=0x0, duration=884.201s, table=0, n_packets=3, n_bytes=306, priority=40000,ip,nw_src=192.168.40.0/24,nw_dst=192.168.30.0/24
actions=drop
cookie=0x0, duration=884.201s, table=0, n_packets=0, n_bytes=0, priority=40000,ip,nw_src=192.168.10.0/24,nw_dst=192.168.20.0/24 a
ctions=drop
cookie=0x0, duration=884.201s, table=0, n_packets=169, n_bytes=24091, priority=0 actions=NORMAL
mininet>

```

Figure 5.11: Flow Table Verification of Packet Interception

### 5.4 Network Traffic Capture and Protocol Interpretation

Network traffic was actively monitored at the core switch interfaces using the Wireshark protocol analyzer to fulfill the deep packet inspection requirements of the project. The following sections interpret the precise academic mechanics of the captured network behaviors.

### 5.4.1 DHCP (Dynamic Host Configuration Protocol)

DHCP is an application-layer protocol utilized to automatically assign IP addresses and network parameters to client devices, reducing administrative overhead. The packet capture below perfectly illustrates the standard four-step "DORA" exchange required to acquire a lease.

#### Interpretation of Capture:

1. **Discover:** The client (Host) has no IP address. It sends a broadcast packet (Destination IP 255.255.255.255) over UDP Port 67 asking "Is there a DHCP server on this network?"
2. **Offer:** The central Core Router receives the broadcast and replies over UDP Port 68, offering an available IP address from the specific VLAN pool.
3. **Request:** The client formally requests permission to use the offered IP address.
4. **ACK:** The router acknowledges the request, binding the IP to the client's MAC address for a set lease time.

```
mininet> h1 dhclient -r h1-eth0
Removed stale PID file
mininet> h1 dhclient -v h1-eth0
Internet Systems Consortium DHCP Client 4.4.3-P1
Copyright 2004-2022 Internet Systems Consortium.
All rights reserved.
For info, please visit https://www.isc.org/software/dhcp/

Listening on LPF/h1-eth0/be:4c:f2:56:1b:fc
Sending on   LPF/h1-eth0/be:4c:f2:56:1b:fc
Sending on   Socket/fallback
xid: warning: no netdev with useable HWADDR found for seed's uniqueness enforcement
xid: rand init seed (0x6a467e01) built using gethostid
DHCPDISCOVER on h1-eth0 to 255.255.255.255 port 67 interval 3 (xid=0x25b37033)
DHCPOFFER of 192.168.10.147 from 192.168.10.1
DHCPREQUEST for 192.168.10.147 on h1-eth0 to 255.255.255.255 port 67 (xid=0x3370b325)
DHCPACK of 192.168.10.147 from 192.168.10.1 (xid=0x25b37033)
Error: ipv4: Address already assigned.
bound to 192.168.10.147 -- renewal in 20469 seconds.
```

Figure 5.12: Terminal Execution of DHCP Client Renewal

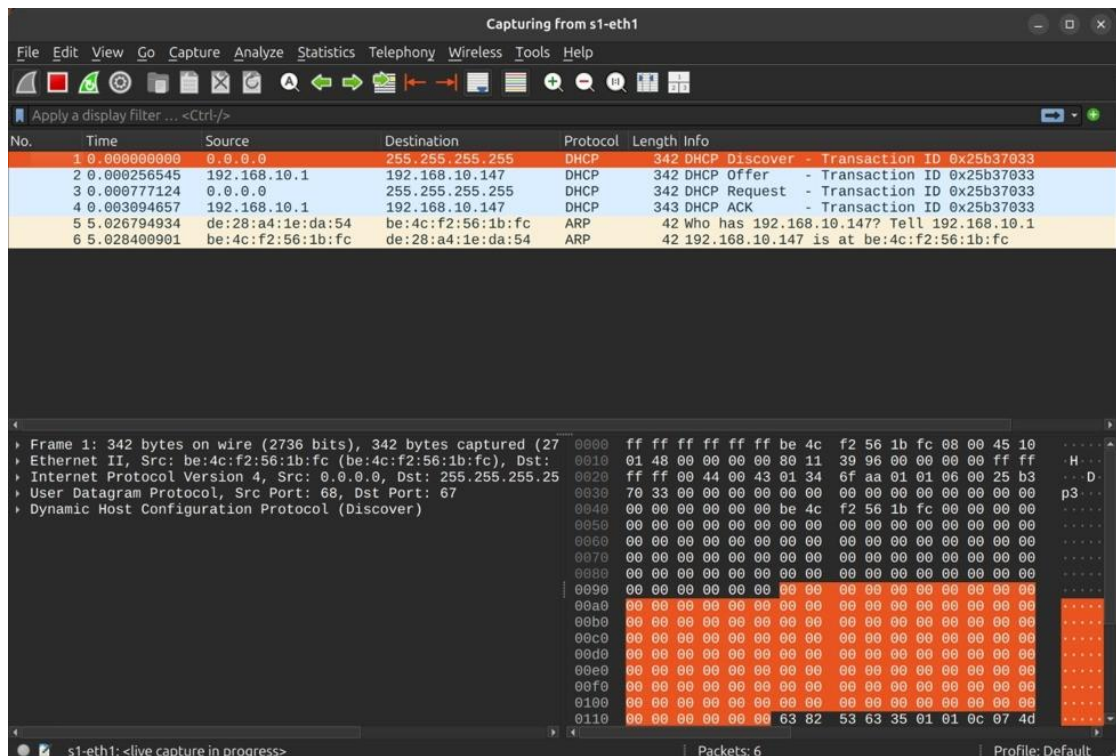


Figure 5.13: Wireshark Capture: DHCP DORA Sequence

## 5.4.2 TCP Retransmission and Firewall Verification

To prove the SDN firewall blocks higher-layer protocols (not just basic ICMP pings), an HTTP connection attempt was made from the blocked Lab network to the Admin server.

**Interpretation of Capture:** Transmission Control Protocol (TCP) guarantees reliable data delivery. When the Lab host attempts to establish a connection, it sends an initial SYN (Synchronize) packet. However, the SDN controller identifies this packet as violating the ACL rule and silently drops it at the switch hardware level. Because the Lab host never receives an acknowledgment (ACK), its internal timer expires. Assuming the packet was simply lost in transit, the host continuously duplicates and resends the SYN packet. This behavior, clearly visible in Figure 5.15 as a series of black TCP Re-transmissions with exponentially increasing delays, conclusively proves the connection is being actively severed by the firewall.

```

mininet> h3 python3 -m http.server 80 &
mininet> h4 curl http://192.168.30.10 --connect-timeout 5
bash: curl: command not found
mininet> h4 curl http://192.168.30.10 --connect-timeout 5
curl: (28) Connection timed out after 5002 milliseconds
mininet> h4 curl http://192.168.30.10 --connect-timeout 5
curl: (28) Connection timed out after 5002 milliseconds
mininet>

```

Figure 5.14: Terminal output showing TCP Connection Timeout

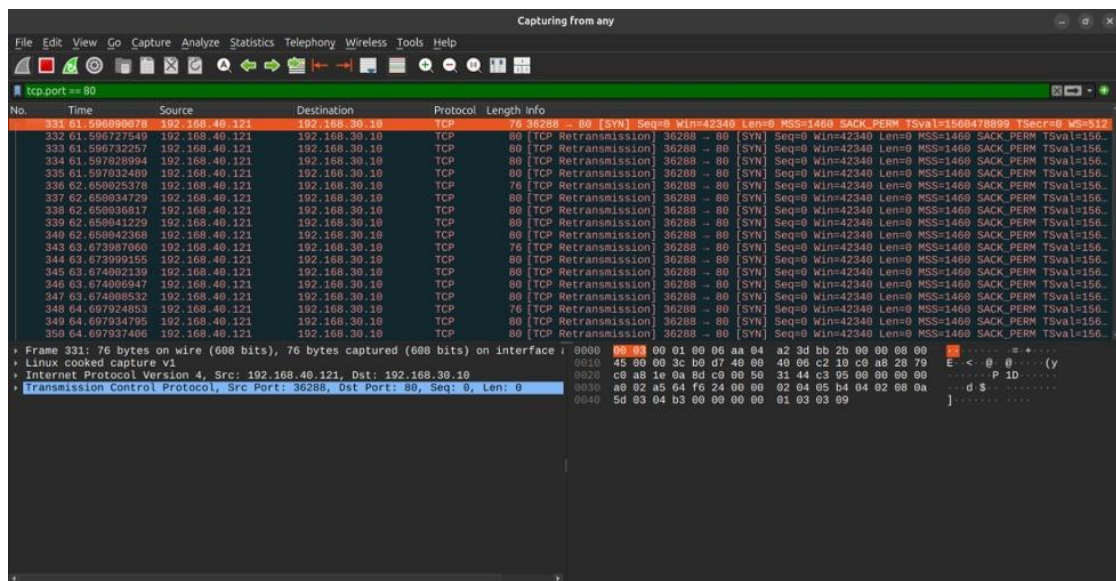


Figure 5.15: Wireshark Capture: TCP Retransmission due to SDN Drop Rule

### 5.4.3 ARP (Address Resolution Protocol)

Before any two devices can communicate over an Ethernet network, they must know each other's physical hardware addresses. ARP is the essential bridge that translates logical Layer-3 IP addresses into physical Layer-2 MAC addresses.

**Interpretation of Capture:** By intentionally flushing the host's IP neighbor table, we forced the generation of fresh ARP traffic. The capture in Figure 5.17 displays the two-step process:

1. **ARP Request:** Host 1 sends a broadcast frame to the entire VLAN asking, "Who has IP address 192.168.10.1? Tell 192.168.10.147."
2. **ARP Reply:** The gateway router recognizes its own IP, and sends a direct unicast reply stating, "I am 192.168.10.1, and my MAC address is X:X:X:X." Host 1 caches this information, allowing subsequent data packets to flow.



```

mininet> h1 ip -s -s neigh flush all
192.168.10.1 dev h1-eth0 lladdr de:28:a4:1e:da:54 used 2571/2571/2550probes 1 STALE

*** Round 1, deleting 1 entries ***
*** Flush is complete after 1 round ***
mininet> h1 ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: h1-eth0@if284: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether be:4c:f2:56:1b:fc brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 192.168.10.147/24 brd 192.168.10.255 scope global dynamic h1-eth0
        valid_lft 39942sec preferred_lft 39942sec
mininet> h1 ping -c 1 192.168.10.1
PING 192.168.10.1 (192.168.10.1) 56(84) bytes of data:
64 bytes from 192.168.10.1: icmp_seq=1 ttl=64 time=2.95 ms

--- 192.168.10.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 2.951/2.951/2.951/0.000 ms
mininet>

```

Figure 5.16: Flushing ARP Cache and initiating traffic

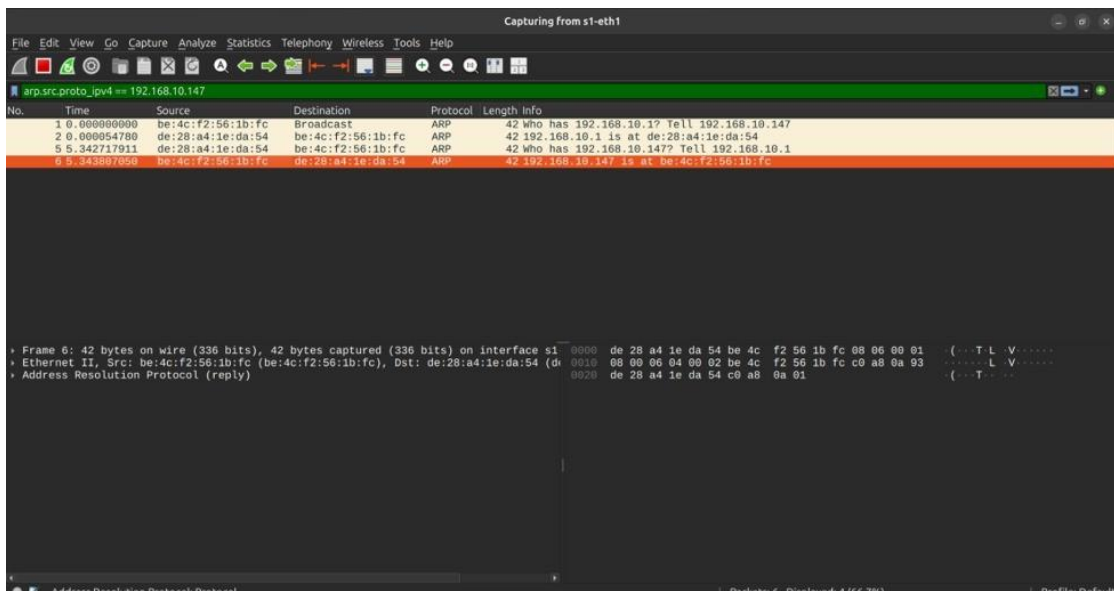


Figure 5.17: Wireshark Capture: ARP Request and Reply

## 5.4.4 TCP Three-Way Handshake

Unlike UDP, TCP is a connection-oriented protocol. It requires a formal setup process before any application data (like a webpage or file transfer) can be transmitted.

**Interpretation of Capture:** A python HTTP server was established on the Admin host. When accessed by an authorized ECE host, Wireshark captured the perfect execution of the TCP Three-Way Handshake.

1. **SYN:** The client sends a sequence number to synchronize the connection.

2. **SYN-ACK:** The server receives the SYN, acknowledges it (ACK), and sends its own sequence number (SYN).
3. **ACK:** The client acknowledges the server's sequence number. The reliable connection is now fully established, and HTTP data begins to transfer immediately afterward.

```
mininet> h3 python3 -m http.server 80 &
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
mininet> h1 curl http://192.168.30.10
<!DOCTYPE HTML>
<html lang="en">
<head>
<meta charset="utf-8">
<title>Directory listing for /</title>
</head>
<body>
<h1>Directory listing for /</h1>
<hr>
<ul>
<li><a href="..lock.Instructions.odt%23">..lock.Instructions.odt#</a></li>
<li><a href="campus_traffic.pcap">campus_traffic.pcap</a></li>
<li><a href="campusNetwork.py">campusNetwork.py</a></li>
<li><a href="campusNetworkCopy.py">campusNetworkCopy.py</a></li>
<li><a href="Chat.odt">Chat.odt</a></li>
<li><a href="customRyu.py">customRyu.py</a></li>
<li><a href="customRyu.pyc">customRyu.pyc</a></li>
<li><a href="dhcpDora.pcapng">dhcpDora.pcapng</a></li>
<li><a href="Instructions.odt">Instructions.odt</a></li>
<li><a href="student_traffic.pcap">student_traffic.pcap</a></li>
</ul>
<hr>
```

Figure 5.18: Successful HTTP Request between authorized subnets

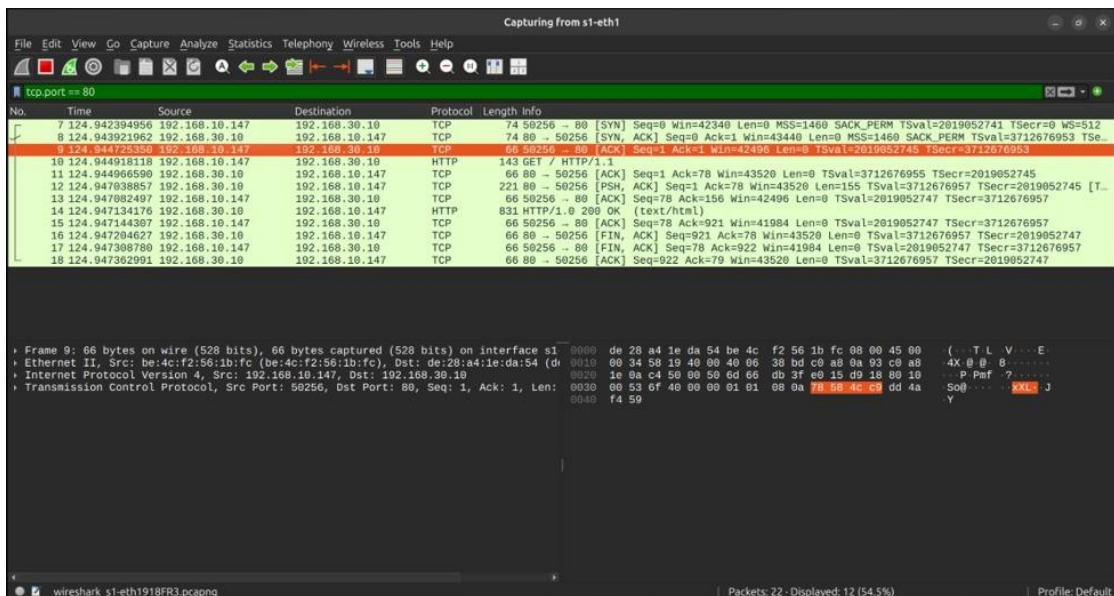


Figure 5.19: Wireshark Capture: TCP Three-Way Handshake

## 5.4.5 DNS (Domain Name System)

DNS operates as the "phonebook" of the network, translating human-readable host-names (like admin.cust.local) into machine-routable IP addresses (like 192.168.30.10).

**Interpretation of Capture:** Figure 5.21 demonstrates the generation of a standard DNS query. The client sends a packet over UDP Port 53 to the centralized DNS resolver asking for the A-Record of the target server. The server successfully replies with the mapped IP address, demonstrating seamless internal service discovery within the isolated campus network.

```
mininet> r0 tcpdump -i r0-eth1 udp port 53 -w /tmp/dns_capture.pcap &
mininet> h4 nslookup labs.cust.local 192.168.10.1
Server:      192.168.10.1
Address:     192.168.10.1#53

Name:   labs.cust.local
Address: 192.168.40.1
```

Figure 5.20: Generating a DNS Request to the Local Resolver

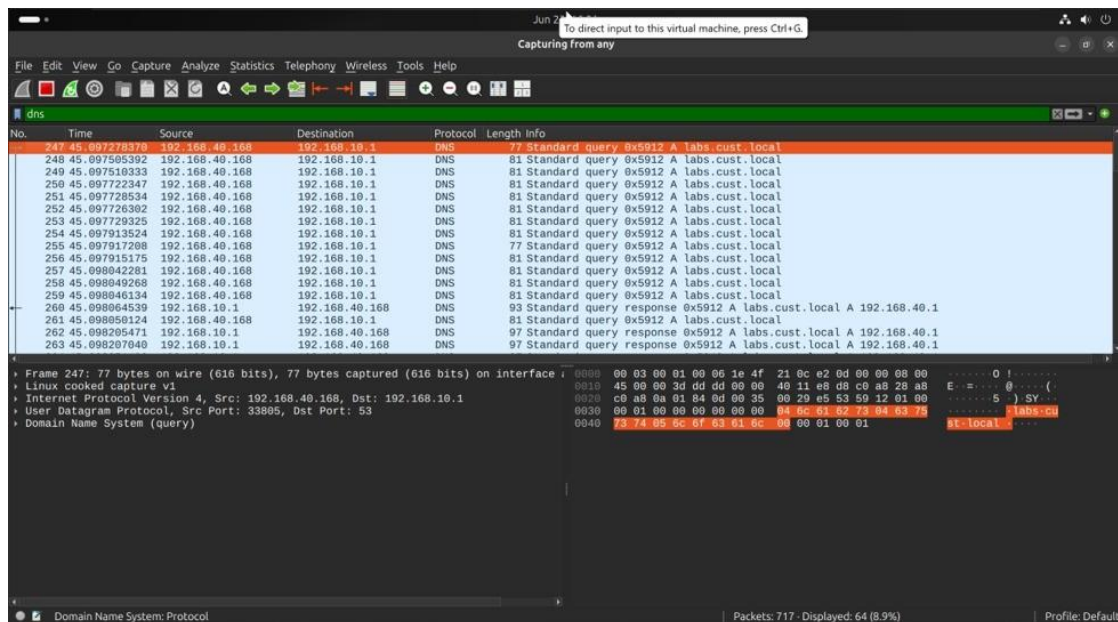


Figure 5.21: Wireshark Capture: DNS Query and Response

## 5.4.6 OSPF Routing Verification

To demonstrate the functionality of dynamic routing protocols within the hierarchical architecture, the OSPF processes were inspected within the FRRouting (FRR) terminal shell. Figures 5.22 and 5.23 verify the active OSPF daemon generating Link State

Advertisements (LSAs) and successfully mapping the internal subnets to the backbone routing table (Area 0). Figure 5.24 displays the running configuration, validating the programmatic integration of the routing paths.

```
mininet> r0 vtysh -c 'show ip ospf'
OSPF Routing Process, Router ID: 192.168.40.1
Supports only single TOS (TOS0) routes
This implementation conforms to RFC2328
RFC1583Compatibility flag is disabled
OpaqueCapability flag is disabled
Initial SPF scheduling delay 0 millisec(s)
Minimum hold time between consecutive SPF's 50 millisec(s)
Maximum hold time between consecutive SPF's 5000 millisec(s)
Hold time multiplier is currently 1
SPF algorithm last executed 2m06s ago
Last SPF duration 266 usecs
SPF timer is inactive
LSA minimum interval 5000 msecs
LSA minimum arrival 1000 msecs
Write Multiplier set to 20
Refresh timer 10 secs
Maximum multiple paths(ECMP) supported 256
Administrative distance 110
Number of external LSA 0. Checksum Sum 0x00000000
Number of opaque AS LSA 0. Checksum Sum 0x00000000
Number of areas attached to this router: 1
Area ID: 0.0.0.0 (Backbone)
  Number of interfaces in this area: Total: 4, Active: 4
  Number of fully adjacent neighbors in this area: 0
```

Figure 5.22: Verification of Active OSPF Process via vtysh



```

SPF timer is inactive
LSA minimum interval 5000 msecs
LSA minimum arrival 1000 msecs
Write Multiplier set to 20
Refresh timer 10 secs
Maximum multiple paths(ECMP) supported 256
Administrative distance 110
Number of external LSA 0. Checksum Sum 0x00000000
Number of opaque AS LSA 0. Checksum Sum 0x00000000
Number of areas attached to this router: 1
Area ID: 0.0.0.0 (Backbone)
  Number of interfaces in this area: Total: 4, Active: 4
  Number of fully adjacent neighbors in this area: 0
  Area has no authentication
  SPF algorithm executed 1 times
  Number of LSA 1
  Number of router LSA 1. Checksum Sum 0x00003d87
  Number of network LSA 0. Checksum Sum 0x00000000
  Number of summary LSA 0. Checksum Sum 0x00000000
  Number of ASBR summary LSA 0. Checksum Sum 0x00000000
  Number of NSSA LSA 0. Checksum Sum 0x00000000
  Number of opaque link LSA 0. Checksum Sum 0x00000000
  Number of opaque area LSA 0. Checksum Sum 0x00000000

```

Figure 5.23: OSPF Link State and Area 0 Verification

```

mininet> r0 vtysh -c 'show running-config'
Building configuration...

Current configuration:
!
frr version 10.4.1
frr defaults traditional
hostname MR-U-VM
log syslog informational
service integrated-vtysh-config
!
router ospf
  network 192.168.0.0/16 area 0
exit
!
end
mininet>

```

Figure 5.24: FRR Running Configuration confirming OSPF Integration

## CHAPTER 6

### CONCLUSION AND FUTURE WORK

This project successfully analyzed real-world TCP/IP behaviors and addressed the severe security and scalability vulnerabilities inherent in legacy flat Layer-2 networks. By simulating a complex enterprise topology using Mininet, the research validated the transition to an SDN-enabled Three-Tier architecture. The integration of 802.1Q VLANs effectively mitigated broadcast storms and localized traffic, while the centralized Ryu controller enforced deterministic, Zero-Trust Access Control Lists across departmental boundaries.

The comprehensive Wireshark analysis verified the stable operation and underlying mechanics of core networking protocols within the virtualized environment. Furthermore, OSPF integration ensured the architecture supports dynamic routing convergence, and flow-table statistics provided a robust centralized monitoring mechanism. Ultimately, this systematic redesign provides a highly scalable, secure, and easily man-ageable infrastructure model that directly fulfills the smart-campus innovation and sustainability objectives outlined in SDG-9 and SDG-11.

## REFERENCES

- [1] McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S., & Turner, J. (2008). OpenFlow: Enabling innovation in campus networks. *ACM SIGCOMM Computer Communication Review*, 38(2), 69-74.
- [2] Lantz, B., Heller, B., & McKeown, N. (2010). A network in a laptop: rapid prototyping for software-defined networks. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets-IX)*.
- [3] Nippon Telegraph and Telephone Corporation. (2020). Ryu SDN Framework. Retrieved from <https://ryu-sdn.org/>
- [4] Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8th ed.). Pearson.
- [5] Moy, J. (1998). OSPF Version 2. IETF RFC 2328. Network Working Group.

# CHAPTER A

## SOURCE CODE

### A.1 Ryu SDN Controller Script (customRyu.py)

Listing A.1: Ryu Controller and OpenFlow ACL Rules

```
1 from ryu.base import app_manager
2 from ryu.controller import ofp_event
3 from ryu.controller.handler import CONFIG_DISPATCHER
4 from ryu.controller.handler import set_ev_cls
5 from ryu.ofproto import ofproto_v1_3
6
7 class CustomCampusController ( app_manager . RyuApp ):
8     # Strictly enforce OpenFlow 1.3
9     OFP_VERSIONS = [ ofproto_v1_3 . OFP_VERSION ]
10
11     def __init__(self, *args, **kwargs):
12         super(CustomCampusController, self).__init__(*args, **kwargs)
13
14     @set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
15     def switch_features_handler(self, ev):
16         datapath = ev.msg.datapath
17         ofproto = datapath.ofproto
18         parser = datapath.ofproto_parser
19
20         # 1. THE HYBRID SDN FIX:
21         # Push a default table-miss flow to all connecting switches.
22         # This tells the switch to use its native OVS pipeline (respecting VLAN
23         # tags)
24         # for any traffic that doesn't match our specific security drops.
25         match = parser.OFPMatch()
26         actions = [parser.OFPActionOutput(ofproto.OFPP_NORMAL)]
27         self.add_flow(datapath, 0, match, actions)
28
29         self.logger.info("Switch fully connected and NORMAL flow pushed : %s",
30                          datapath.id)
31
32         # 2. Apply Custom Security Policies ONLY to the Core Switch (s1)
33         if datapath.id == 1:
34             self.logger.info("Applying CUST campus ACL policy to Core Switch (s1)
35                             ...")
36             self.apply_custom_firewall(datapath)
37
38     def add_flow(self, datapath, priority, match, actions):
39         ofproto = datapath.ofproto
40         parser = datapath.ofproto_parser
```

```

39     inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS , actions
40         )]
41     mod = parser.OFPFlowMod(datapath=datapath , priority=priority ,
42         match=match , instructions=inst)
43     datapath.send_msg(mod)
44
45 def apply_custom_firewall(self , datapath):
46     parser = datapath.ofproto_parser
47
48     # Rule 1: Student Labs (40) to Admin (30) Drop
49     match1 = parser.OFPMatch(
50         eth_type=0x0800 ,
51         ipv4_src='192.168.40.0/255.255.255.0' ,
52         ipv4_dst='192.168.30.0/255.255.255.0')
53     self.add_flow(datapath , 40000 , match1 , [])
54     self.logger.info("ACL_Rule_1_installed:_Student_Labs (40)_->_Admin (30)_
55         DROP")
56
57     # Rule 2: ECE (10) to CS (20) Drop
58     match2 = parser.OFPMatch(
59         eth_type=0x0800 ,
60         ipv4_src='192.168.10.0/255.255.255.0' ,
61         ipv4_dst='192.168.20.0/255.255.255.0')
62     self.add_flow(datapath , 40000 , match2 , [])
63     self.logger.info("ACL_Rule_2_installed:_ECE_Dept (10)_->_CS_Dept (20)_DROP"
64         )

```

## A.2 Mininet Topology Script (campusNetwork.py)

Listing A.2: Mininet Topology and Host Configuration

```
1 import time
2 import os
3 from mininet.topo import Topo
4 from mininet.net import Mininet
5 from mininet.node import Node, RemoteController
6 from mininet.cli import CLI
7 from mininet.log import setLogLevel
8
9
10 class OSPFRouter ( Node ):
11     def config( self, ** params ):
12         super( OSPFRouter , self ). config ( ** params )
13         self.cmd ( ' sysctl_net.ipv4.ip_forward = 1 ' )
14
15     def start_ospf( self ):
16         # 1. Fix the Linux folder permission crashes
17         self.cmd ( ' mkdir_p _p_ / var / run / frr ' )
18         self.cmd ( ' chown _R_root: root_ / var / run / frr_2 > / dev / null ' )
19         self.cmd ( ' chmod _777 _ / var / run / frr ' )
20
21         # 2. Create isolated config files
22         zebra_conf = " / tmp / zebra_r0 . conf "
23         ospfd_conf = " / tmp / ospfd_r0 . conf "
24
25         with open( zebra_conf , 'w' ) as f:
26             f.write ( " hostname_r0 \ n password_zebra \ n " )
27
28         with open( ospfd_conf , 'w' ) as f:
29             f.write ( " hostname_r0 \ n password_zebra \ n " )
30             f.write ( " router_ospf \ n " )
31             f.write ( " _ospf_router - id_ 192.168.10.1 \ n " )
32             f.write ( " _network _ 192.168.10.0 / 24 _area _ 0 \ n " )
33             f.write ( " _network _ 192.168.20.0 / 24 _area _ 0 \ n " )
34             f.write ( " _network _ 192.168.30.0 / 24 _area _ 0 \ n " )
35             f.write ( " _network _ 192.168.40.0 / 24 _area _ 0 \ n " )
36
37         # 3. Kill existing dead instances
38         self.cmd ( ' pkill _9_ - f_ospfd_2 > / dev / null ' )
39         self.cmd ( ' pkill _9_ - f_zebra_2 > / dev / null ' )
40
41         # 4. Launch Daemons AS ROOT using the standard system socket
42         print ( " _ _ _ _ _ Starting_Zebra_core_routing_engine ... " )
43         self.cmd ( ' / usr / lib / frr / zebra _ - f_ { } _ - i_ / var / run / frr / zebra . pid _ - z_ / var / run /
44             frr / zserv . api _ - d _ - A_ 127.0.0.1 _ - u_root _ - g_root ' . format ( zebra_conf ) )
45         time . sleep ( 1 )
46
47         print ( " _ _ _ _ _ Starting _OSPF _routing_engine ... " )
48         self.cmd ( ' / usr / lib / frr / ospfd _ - f_ { } _ - i_ / var / run / frr / ospfd . pid _ - z_ / var / run /
49             frr / zserv . api _ - d _ - A_ 127.0.0.1 _ - u_root _ - g_root ' . format ( ospfd_conf ) )
50
51     def terminate ( self ):
52         self.cmd ( ' pkill _9_ - f_ospfd ' )
53         self.cmd ( ' pkill _9_ - f_zebra ' )
```

```

52     self.cmd('sysctl net.ipv4.ip_forward=0')
53     super(OSPFRouter, self).terminate()
54
55
56 class CustCampusTopo(Topo):
57     def build(self):
58         r0 = self.addNode('r0', cls=OSPFRouter)
59         s1_core = self.addSwitch('s1')
60
61         s2_dist_east = self.addSwitch('s2')
62         s3_dist_west = self.addSwitch('s3')
63
64         s4_access_ece = self.addSwitch('s4')
65         s5_access_cs = self.addSwitch('s5')
66         s6_access_admin = self.addSwitch('s6')
67         s7_access_labs = self.addSwitch('s7')
68
69         h1_ece_pc = self.addHost('h1', ip=None)
70         h2_cs_pc = self.addHost('h2', ip=None)
71         h3_admin_pc = self.addHost('h3', ip=None)
72         h4_lab_pc = self.addHost('h4', ip=None)
73
74         self.addLink(r0, s1_core, intfName1='r0-eth1', params1={'ip': '
75             192.168.10.1/24'})
76         self.addLink(r0, s1_core, intfName1='r0-eth2', params1={'ip': '
77             192.168.20.1/24'})
78         self.addLink(r0, s1_core, intfName1='r0-eth3', params1={'ip': '
79             192.168.30.1/24'})
80         self.addLink(r0, s1_core, intfName1='r0-eth4', params1={'ip': '
81             192.168.40.1/24'})
82
83         self.addLink(s1_core, s2_dist_east)
84         self.addLink(s1_core, s3_dist_west)
85
86         self.addLink(s2_dist_east, s4_access_ece)
87         self.addLink(s2_dist_east, s5_access_cs)
88         self.addLink(s3_dist_west, s6_access_admin)
89         self.addLink(s3_dist_west, s7_access_labs)
90
91         self.addLink(h1_ece_pc, s4_access_ece)
92         self.addLink(h2_cs_pc, s5_access_cs)
93         self.addLink(h3_admin_pc, s6_access_admin)
94         self.addLink(h4_lab_pc, s7_access_labs)
95
96 def real_port_name(net, switch_name, peer_node_name, peer_intf_hint=None):
97     switch = net[switch_name]
98     for intf in switch.intfList():
99         if intf.name == 'lo':
100             continue
101         link = intf.link
102         if link is None:
103             continue
104         other_intf = link.intf1 if link.intf2 == intf else link.intf2
105         if other_intf is None or other_intf.node is None:
106             continue
107         if other_intf.node.name == peer_node_name:
108             if peer_intf_hint is None or peer_intf_hint == other_intf.name:

```

```

106         return intf.name
107     return None
108
109
110 def tag_access_port(net, switch_name, intf_name, vlan_id, label=''):
111     if intf_name is None:
112         return False
113     net[ switch_name ].cmd('ovs -vsctl set_port_{0} tag={1}'.format(intf_name, vlan_id))
114     print("_{0}_{1} >3}_{2}_{3} <10}_{4}->_ACCESS_vlan {5} <4}_{6}".format(switch_name,
115                                     intf_name, vlan_id, label))
116     return True
117
118 def tag_trunk_port(net, switch_name, intf_name, vlan_ids, label=''):
119     if intf_name is None:
120         return False
121     vlans = ','.join(str(v) for v in vlan_ids)
122     net[ switch_name ].cmd('ovs -vsctl set_port_{0} trunks={1}'.format(intf_name,
123                                     vlans))
124     print("_{0}_{1} >3}_{2}_{3} <10}_{4}->_TRUNK_{5}_{6}".format(switch_name, intf_name,
125                                     vlans, label))
126     return True
127
128 def setup_vlans(network):
129     print("\n***_Applying_802.1Q_VLAN_tagging_and_enforcing_OpenFlow_1.3...")
130
131     for sw in network.switches:
132         sw.cmd('ovs -vsctl set_bridge_{0} protocols = OpenFlow13'.format(sw.name))
133         sw.cmd('ovs -vsctl set -fail -mode_{0}_secure'.format(sw.name))
134
135     VLAN_ECE, VLAN_CS, VLAN_ADMIN, VLAN_LABS = 10, 20, 30, 40
136     ALL_VLANS = [VLAN_ECE, VLAN_CS, VLAN_ADMIN, VLAN_LABS]
137
138     tag_access_port(network, 's1', real_port_name(network, 's1', 'r0', 'r0-eth1'),
139                     VLAN_ECE, 'r0-eth1/_ECE')
140     tag_access_port(network, 's1', real_port_name(network, 's1', 'r0', 'r0-eth2'),
141                     VLAN_CS, 'r0-eth2/_CS')
142     tag_access_port(network, 's1', real_port_name(network, 's1', 'r0', 'r0-eth3'),
143                     VLAN_ADMIN, 'r0-eth3/_Admin')
144     tag_access_port(network, 's1', real_port_name(network, 's1', 'r0', 'r0-eth4'),
145                     VLAN_LABS, 'r0-eth4/_Labs')
146
147     tag_trunk_port(network, 's1', real_port_name(network, 's1', 's2'), ALL_VLANS,
148                     's1->s2')
149     tag_trunk_port(network, 's1', real_port_name(network, 's1', 's3'), ALL_VLANS,
150                     's1->s3')
151
152     tag_trunk_port(network, 's2', real_port_name(network, 's2', 's1'), ALL_VLANS,
153                     's2->s1')
154     tag_trunk_port(network, 's2', real_port_name(network, 's2', 's4'), ALL_VLANS,
155                     's2->s4')
156     tag_trunk_port(network, 's2', real_port_name(network, 's2', 's5'), ALL_VLANS,
157                     's2->s5')
158
159     tag_trunk_port(network, 's3', real_port_name(network, 's3', 's1'), ALL_VLANS,
160                     's3->s1')

```



```

150 tag_trunk_port(network, 's3', real_port_name(network, 's3', 's6'), ALL_VLANS,
    's3->s6')
151 tag_trunk_port(network, 's3', real_port_name(network, 's3', 's7'), ALL_VLANS,
    's3->s7')
152
153 tag_trunk_port(network, 's4', real_port_name(network, 's4', 's2'), ALL_VLANS,
    's4->s2')
154 tag_access_port(network, 's4', real_port_name(network, 's4', 'h1'), VLAN_ECE,
    's4->h1')
155
156 tag_trunk_port(network, 's5', real_port_name(network, 's5', 's2'), ALL_VLANS,
    's5->s2')
157 tag_access_port(network, 's5', real_port_name(network, 's5', 'h2'), VLAN_CS,
    's5->h2')
158
159 tag_trunk_port(network, 's6', real_port_name(network, 's6', 's3'), ALL_VLANS,
    's6->s3')
160 tag_access_port(network, 's6', real_port_name(network, 's6', 'h3'),
    VLAN_ADMIN, 's6->h3')
161
162 tag_trunk_port(network, 's7', real_port_name(network, 's7', 's3'), ALL_VLANS,
    's7->s3')
163 tag_access_port(network, 's7', real_port_name(network, 's7', 'h4'), VLAN_LABS,
    's7->h4')
164
165
166 def start_dhcp_services(network):
167     """
168     ...THREE SEPARATE dnsmasq instances, one per VLAN interface.
169
170     ...DNS_UPDATE: the 'ece' instance now also serves internal DNS (port = 53
171     ...instead of port = 0), since DNS lookups aren't subnet-bound the way
172     ...DHCP broadcasts are - any host that can route to r0-eth1's IP can
173     ...query it for names, regardless of which VLAN that host is on. This
174     ...gives the campus internal name resolution (e.g. admin.cust.local)
175     ...without needing NAT / internet access, which is intentionally out of
176     ...scope for this simulation. All three pools' dhcp-option = 6 (DNS
177     ...server) now point at 192.168.10.1, where DNS is actually listening,
178     ...instead of the old 192.168.30.10 (Admin host, which never ran DNS).
179     """
180     r0 = network['r0']
181     r0.cmd('pkill dnsmasq 2>/dev/null')
182     time.sleep(0.5)
183
184     configs = {
185         'ece': {
186             'iface': 'r0-eth1', 'conf': '/tmp/dnsmasq_ece.conf', 'pid': '/tmp/
                dnsmasq_ece.pid',
187             'body': ("port = 53\ninterface = r0-eth1\nbind - interfaces\nexcept -
                interface = lo\nndhcp - authoritative\n"
188                     "dhcp - range = 192.168.10.100,192.168.10.200,255.255.255.0,12 h\
                n"
189                     "dhcp - option = 3,192.168.10.1\nndhcp - option = 6,192.168.10.1\n"
190                     "no-resolv\n"
191                     "domain = cust.local\n"
192                     "expand - hosts\n"
193                     "address = /admin.cust.local/192.168.30.10\n"
194                     "address = /ece.cust.local/192.168.10.1\n")

```

```

195         "address = / cs.cust.local / 192.168.20.1 \n"
196         "address = / labs.cust.local / 192.168.40.1 \n"
197         "address = / portal.cust.local / 192.168.30.10 \n"),
198     },
199     'cs': {
200         'iface': 'r0-eth2', 'conf': '/tmp/dnsmasq_cs.conf', 'pid': '/tmp/
201             dnsmasq_cs.pid',
202         'body': ("port = 0 \n interface = r0-eth2 \n bind - interfaces \n except -
203             interface = lo \n dhcp - authoritative \n"
204                 "dhcp - range = 192.168.20.100,192.168.20.200,255.255.255.0,12 h\
205                 n"
206                 "dhcp - option = 3,192.168.20.1 \n dhcp - option = 6,192.168.10.1 \n"),
207     },
208     'labs': {
209         'iface': 'r0-eth4', 'conf': '/tmp/dnsmasq_labs.conf', 'pid': '/tmp/
210             dnsmasq_labs.pid',
211         'body': ("port = 0 \n interface = r0-eth4 \n bind - interfaces \n except -
212             interface = lo \n dhcp - authoritative \n"
213                 "dhcp - range = 192.168.40.100,192.168.40.200,255.255.255.0,12 h\
214                 n"
215                 "dhcp - option = 3,192.168.40.1 \n dhcp - option = 6,192.168.10.1 \n"),
216     },
217 }
218
219 for name, cfg in configs.items():
220     with open(cfg['conf'], 'w') as f:
221         f.write(cfg['body'])
222     r0.cmd('dnsmasq -C {conf} --pid - file = {pid}_2 >&1'.format(**cfg))
223     time.sleep(0.5)
224     pid_check = r0.cmd('cat {pid}_2 >/dev/null'.format(cfg['pid'])).strip()
225     if pid_check and pid_check.isdigit():
226         print("{}_dnsmasq_for_{}_VLAN_bound_to_{}_({pid})".format(name,
227             cfg['iface'], pid_check))
228
229 time.sleep(2)
230
231 def request_dhcp_lease(network, host_name, iface, expected_subnet, gateway):
232     host = network[host_name]
233
234     for attempt in range(2):
235         host.cmd('pkill -9 -f "dhclient.*{}" >/dev/null'.format(iface))
236         host.cmd('ip_addr_flush_dev {} >/dev/null'.format(iface))
237         time.sleep(1)
238
239         cmd = 'timeout 15 dhclient -f /tmp/{0}.leases -pf /tmp/{0}.pid -v {1} >
240             /tmp/{0}_dhclient.log >&1'.format(host_name, iface)
241         host.cmd(cmd)
242         time.sleep(1.5)
243
244         ip_output = host.cmd('ip -4 addr show {}'.format(iface))
245         if 'inet' in ip_output:
246             ip_line = [l.strip() for l in ip_output.splitlines() if 'inet' in l
247                 ][0]
248             leased_ip = ip_line.split()[1].split('/')[0]
249
250             if expected_subnet in leased_ip:

```

```

243         print("{}-{}_leased:{}_({}_correct_VLAN_pool)".format(host_name,
244                               ip_line))
245         return True
246     else:
247         print("{}!!_VLAN_LEAK:{}_leased{}_but_expected{}.x".format(
248             host_name, leased_ip, expected_subnet))
249         return False
250
251     if attempt == 0:
252         print("{}-{}_DHCP_attempt_1_timed_out._Retrying_with_clean_state
253         ...".format(host_name))
254
255     print("{}!!_WARNING:{}_did_NOT_receive_a_DHCP_lease._Check_/tmp/{}_
256     _dhclient.log".format(host_name, host_name))
257     return False
258
259 def execute_network():
260     topology = CustCampusTopo()
261     network = Mininet(topo=topology, controller=RemoteController)
262     network.start()
263
264     print("\n***_[]/5]_Core_router_interfaces_are_up")
265
266     r0 = network['r0']
267     r0.cmd('ifconfig_r0-eth1_192.168.10.1_netmask_255.255.255.0_up')
268     r0.cmd('ifconfig_r0-eth2_192.168.20.1_netmask_255.255.255.0_up')
269     r0.cmd('ifconfig_r0-eth3_192.168.30.1_netmask_255.255.255.0_up')
270     r0.cmd('ifconfig_r0-eth4_192.168.40.1_netmask_255.255.255.0_up')
271
272     print("***_[]1.5/5]_Launching_isolated_OSPF_Daemons_inside_r0...")
273     r0.start_ospf()
274
275     for node in network.values():
276         for intf in node.intfList():
277             if intf.name != 'lo':
278                 node.cmd('ethtool--offload{}_tx_off_rx_off_2>/dev/null'.format(
279                     intf.name))
280
281     setup_vlans(network)
282
283     print("\n***_Waiting_5_seconds_for_Ryu_Controller_to_synchronize_with_all_
284     switches...")
285     time.sleep(5)
286
287     print("\n***_[]2/5]_Starting_centralized_DHCP_service_on_r0_(per-VLAN)...")
288     start_dhcp_services(network)
289
290     print("***_[]3/5]_Assigning_static_service_IP_to_Admin_host_(h3)...")
291     network['h3'].setIP('192.168.30.10', 24)
292     network['h3'].cmd('route_add_default_gw_192.168.30.1')
293
294     print("***_[]4/5]_Requesting_DHCP_leases_for_h1_(ECE),h2_(CS),h4_(Labs)...")
295     request_dhcp_lease(network, 'h1', 'h1-eth0', '192.168.10.', '192.168.10.1')
296     request_dhcp_lease(network, 'h2', 'h2-eth0', '192.168.20.', '192.168.20.1')
297     request_dhcp_lease(network, 'h4', 'h4-eth0', '192.168.40.', '192.168.40.1')

```

```

294     print("\n *** _Network _is_up_with _true _isolated _OSPF _Routing _and _internal_DNS _(
          cust.local).")
295     print(" _ _ _ _ _ Try : _h2 _nslookup _admin .cust.local")
296     print(" _ _ _ _ _ _ _ _ _ _ h4 _nslookup _labs .cust.local")
297     CLI(network)
298
299     network ['r0'].cmd (' pkill _dnsmasq _2 > / dev / null ')
300     network .stop ()
301
302
303 if __name__ == '__main__':
304     setLogLevel (' info ')
305     os.system ("rm _f_ / tmp /* dhcp *_2 > / dev / null")
306     execute_network ()

```